

Unit 3: Similarity-based Learning, Regression Analysis & Decision Trees

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Similarity-based Learning: Instance-based vs Eager learning, KKK-Nearest-Neighbor (KKK-NN) algorithm, distance metrics (Euclidean, Manhattan, Minkowski), Weighted KKK-NN ($1/d_1/d_2$), Nearest Centroid (Rocchio) classifier, Locally Weighted Regression...

Table of Contents

Nearest-Neighbor Learning (KKK-NN)	p. 2
Weighted KKK-Nearest-Neighbor Algorithm	p. 3
Nearest Centroid Classifier (Rocchio)	p. 5
Locally Weighted Regression (LWR)	p. 5
Introduction to Regression Analysis	p. 6
Linear Regression: Ordinary Least Squares (OLS)	p. 7
Polynomial Regression & Bias-Variance Tradeoff	p. 8
Logistic Regression	p. 9
Decision Tree Learning Model	p. 11
Decision Tree Induction Algorithms: ID3, C4.5 & CART	p. 12
Unit 3 Quick Revision Sheet	p. 14
Important Exam Questions	p. 15
Solved Questions from Past Department Examinations	p. 18
Practice Problems (Step-by-Step Verified Solutions)	p. 23

Nearest-Neighbor Learning (KNN-NN)

§ 1

Unlike eager learning algorithms (such as Neural Networks or Decision Trees) that construct an explicit, generalized global model during training, **Nearest-Neighbor methods are instance-based or lazy learners**. They store training instances directly in memory and defer all computation until a query instance arrives.

Definition: K-Nearest Neighbor Algorithm:

Given a query point $\mathbf{x}_q$, K -NN locates the K training instances nearest to $\mathbf{x}_q$ according to a distance metric, and predicts the class label via majority vote:

$$\hat{y}_q = \text{mode}(\{y_i \mid \mathbf{x}_i \in N_K(\mathbf{x}_q)\})$$

For continuous regression targets, it computes the local arithmetic mean:

$$\hat{y}_q = \frac{1}{K} \sum_{\mathbf{x}_i \in N_K(\mathbf{x}_q)} y_i$$

Standard Distance Metrics

- **Euclidean Distance (L_2 norm):** The straight-line geometric distance:

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{i=1}^d (x_i - z_i)^2}$$

- **Manhattan Distance (L_1 norm / City Block):** Sum of absolute coordinate differentials (robust to extreme outliers):

$$d(\mathbf{x}, \mathbf{z}) = \sum_{i=1}^d |x_i - z_i|$$

- **Minkowski Distance (L_p norm):** Generalized metric family:

$$d(\mathbf{x}, \mathbf{z}) = \left(\sum_{i=1}^d |x_i - z_i|^p \right)^{1/p}$$

($p = 1 \implies$ Manhattan; $p = 2 \implies$ Euclidean; $p \rightarrow \infty \implies$ Chebyshev / Max norm).

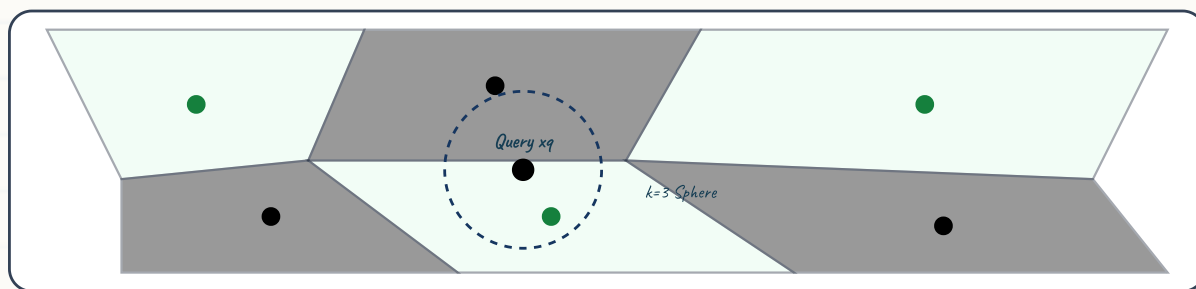


Figure 3.1: Voronoi tessellation: Convex polygonal cells determine nearest-neighbor decision boundaries.

Quick Recall

K -NN is a lazy, instance-based learner. Small K (e.g., $K = 1$) yields low bias but high variance (sensitive to noise); large K yields smooth decision boundaries with higher bias.

Weighted KKK-Nearest-Neighbor Algorithm

§ 2

A major drawback of vanilla K -NN is that all K neighbors cast equal votes, regardless of whether a neighbor is 0.01 units away or 10.0 units away. In **Distance-Weighted K -NN** (Shepard's Method), each neighbor's vote is scaled inversely proportional to the square of its distance:

Distance-Weighted Voting Formulation:

Weight assigned to neighbor $\mathbf{x}_i$:

$$w_i = \frac{1}{d(\mathbf{x}_q, \mathbf{x}_i)^2}$$

(If $d(\mathbf{x}_q, \mathbf{x}_i) = 0$, assign the target label of that exact training point directly).

Weighted Classification:

$$\hat{y}_q = \arg \max_{v \in V} \sum_{i=1}^K w_i \cdot \mathbb{I}(y_i = v)$$

Weighted Regression:

$$\hat{y}_q = \frac{\sum_{i=1}^K w_i y_i}{\sum_{i=1}^K w_i}$$

Solved Problem 1 · Exam Level Instance-Based Learning

Given a 2-dimensional training dataset with $n = 5$ instances:

$$A(1, 2) \rightarrow +1, \quad B(2, 3) \rightarrow +1, \quad C(3, 1) \rightarrow -1, \quad D(5, 4) \rightarrow -1$$

Classify the query point $\mathbf{x}_q = (2, 2)$ using:

1. Unweighted K -NN with $K = 3$.
2. Distance-Weighted K -NN with $K = 3$ using $w_i = \frac{1}{d(\mathbf{x}_q, \mathbf{x}_i)^2}$.

Step 1: Compute Euclidean Distances to Query Point $\mathbf{x}_q(2, 2)$

- $d(Q, A) = \sqrt{(1-2)^2 + (2-2)^2} = \sqrt{(-1)^2 + 0} = \sqrt{1} = 1.0000$ (Label: +1)
- $d(Q, B) = \sqrt{(2-2)^2 + (3-2)^2} = \sqrt{0 + 1^2} = \sqrt{1} = 1.0000$ (Label: +1)
- $d(Q, C) = \sqrt{(3-2)^2 + (1-2)^2} = \sqrt{1^2 + (-1)^2} = \sqrt{2} \approx 1.4142$ (Label: -1)
- $d(Q, D) = \sqrt{(5-2)^2 + (4-2)^2} = \sqrt{3^2 + 2^2} = \sqrt{13} \approx 3.6056$ (Label: -1)
- $d(Q, E) = \sqrt{(6-2)^2 + (5-2)^2} = \sqrt{4^2 + 3^2} = \sqrt{25} = 5.0000$ (Label: -1)

Step 2: Identify $K = 3$ Nearest Neighbors

The 3 closest instances in ascending order are:

Rank 1: Point A ($d = 1.0$, $y = +1$)

Rank 2: Point B ($d = 1.0$, $y = +1$)

Rank 3: Point C ($d = \sqrt{2} \approx 1.4142$, $y = -1$)

Step 3: Unweighted Majority Vote ($K = 3$)

Votes for (+1) : 2 (Points A, B)

Votes for (-1) : 1 (Point C)

$$\hat{y}_q = +1$$

Step 4: Distance-Weighted Voting ($w_i = 1/d_i^2$)

$$w_A = \frac{1}{(1.0)^2} = 1.0000$$

$$w_B = \frac{1}{(1.0)^2} = 1.0000$$

$$w_C = \frac{1}{(1.4142)^2} = 0.5000$$

Quick Recall

Weighted K -NN scales votes by $w_i = 1/d_i^2$, completely neutralizing the influence of distant, uninformative neighbors.

Nearest Centroid Classifier (Rocchio)

§ 3

The **Nearest Centroid Classifier** compresses every training class into a single representative prototype vector (the class centroid) during training, significantly reducing memory and query evaluation costs:

Mathematical Formulation: Nearest Centroid:

For each class $c \in \{1, \dots, C\}$, compute its centroid μ_c :

$$\mu_c = \frac{1}{N_c} \sum_{i:y_i=c} \mathbf{x}_i$$

To classify query point $\mathbf{x}_q$, assign it to the class with the closest centroid:

$$\hat{y}_q = \arg \min_c \|\mathbf{x}_q - \mu_c\|_2$$

Comparison: Nearest Centroid vs. K -NN

Dimension	Nearest Centroid	K -Nearest Neighbors (K -NN)
Decision Boundary	Strictly linear (hyperplanes perpendicular to centroid connectors)	Complex, non-linear, piece-wise polygonal boundaries
Storage Complexity	$O(C \times d)$ (stores only C centroid vectors)	$O(n \times d)$ (stores all n training instances)
Inference Time	Extremely fast: $O(C \times d)$ distance computations	Slow: $O(n \times d)$ distance computations without indexing trees
Non-Convex Classes	Fails if classes are non-convex or multimodal	Accurately captures arbitrary non-convex manifolds

Quick Recall

Nearest Centroid represents each class by its mean vector μ_c . Fast and linear, but fails when classes are non-spherical or multimodal.

Locally Weighted Regression (LWR)

§ 4

Standard linear regression fits a single global straight line across all training data. In contrast, **Locally Weighted Regression (LWR / Loess)** is a non-parametric method that fits a customized local linear model in the immediate neighborhood around each specific query point $\mathbf{x}_q$:

LWR Objective Loss Function:

Find weights θ to minimize the locally weighted squared error:

$$J(\theta) = \sum_{i=1}^n w_i (y_i - \theta^T \mathbf{x}_i)^2$$

Where w_i is a Gaussian-like bell-shaped kernel weighting function centered at query point $\mathbf{x}_q$:

$$w_i = \exp \left(-\frac{\|\mathbf{x}_i - \mathbf{x}_q\|^2}{2\tau^2} \right)$$

Where τ is the **Bandwidth Parameter** controlling the width of the neighborhood:

- If $\mathbf{x}_i$ is close to $\mathbf{x}_q$, $\|\mathbf{x}_i - \mathbf{x}_q\| \rightarrow 0 \implies w_i \rightarrow 1$ (high influence).
- If $\mathbf{x}_i$ is far from $\mathbf{x}_q$, $\|\mathbf{x}_i - \mathbf{x}_q\|$ is large $\implies w_i \rightarrow 0$ (negligible influence).

Closed-form solution for the local parameter vector:

$$\theta = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{y}$$

Where $\mathbf{W}$ is an $n \times n$ diagonal matrix with $\mathbf{W}_{ii} = w_i$.

Quick Recall

LWR is non-parametric: it retrains a local linear model for every query point $\mathbf{x}_q$, weighting instances by Gaussian kernel $\exp(-\|x_i - x_q\|^2 / 2\tau^2)$.

Introduction to Regression Analysis

§ 5

Regression analysis models the functional relationship between one or more **independent explanatory variables** (features $\mathbf{X} = [x_1, \dots, x_d]$) and a **continuous numerical dependent variable** (target Y).

Regression vs. Classification

- **Target Domain:** Classification outputs discrete categorical classes ($Y \in \{0, 1\}$ or $\{C_1, \dots, C_k\}$); Regression outputs real-valued continuous numbers ($Y \in \mathbb{R}$).
- **Evaluation Metrics:** Classification uses Accuracy, Precision, Recall, F_1 , and ROC-AUC; Regression uses Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Coefficient of Determination (R^2).

Quick Recall

Regression predicts continuous numerical quantities ($Y \in \mathbb{R}$). Evaluated via Mean Squared Error (MSE) and R^2 .

Linear Regression: Ordinary Least Squares (OLS)

§ 6

Linear regression assumes a linear relationship between features and target.

Simple Linear Regression (Single Feature)

$$y = mx + c + \epsilon$$

Ordinary Least Squares (OLS) minimizes the Residual Sum of Squares (RSS):

$$RSS = \sum_{i=1}^n (y_i - (mx_i + c))^2$$

Taking partial derivatives with respect to m and c and setting them to zero yields the closed-form solutions:

OLS Closed-Form Formulas:

$$m = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)}$$

$$c = \bar{y} - m\bar{x}$$

Multiple Linear Regression (Matrix Normal Equation)

For d features, $\hat{y} = \mathbf{X}\mathbf{w}$. The closed-form analytical solution that minimizes Mean Squared Error is the **Normal Equation**:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})^2$	$(x_i - \bar{x})(y_i - \bar{y})$	$(y_i - \bar{y})^2$
1	2	-2	-2	4	+4	4
2	3	-1	-1	1	+1	1
3	5	0	+1	0	0	1
4	4	+1	0	1	0	0
5	6	+2	+2	4	+4	4
Total		0	0	10	9	10

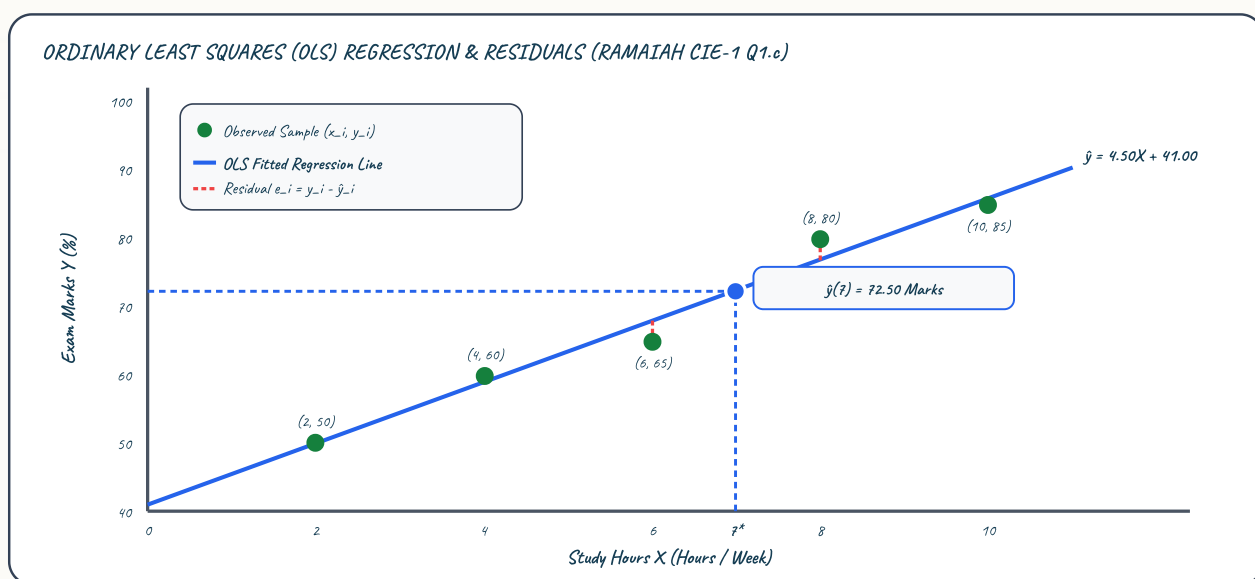


Figure 3.1: Ordinary Least Squares (OLS) Regression Fit & Residuals (Ramaiah CIE-1 Q1.c Solved)

Study Hours (X) vs Exam Marks (Y) yields optimal least-squares line $\hat{Y} = 4.50X + 41.00$. Vertical red lines depict residuals $e_i = y_i - \hat{y}_i$. Target prediction for $X = 7$ hours is $\hat{Y}(7) = 72.50$ marks.

 **Interactive Tool Available Online:**  [Interactive Linear Regression & Error Minimizer](#). Explore the step-by-step calculator and visualizer on the live site notes page.

Quick Recall

OLS slope $m = \frac{\text{Cov}(X,Y)}{\text{Var}(X)}$ and $c = \bar{y} - m\bar{x}$. Matrix Normal Equation: $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. $R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$.

Polynomial Regression & Bias-Variance Tradeoff

When the relationship between features and target is non-linear (e.g., parabolic or cyclical), fitting a straight line causes severe **underfitting** (high bias). **Polynomial Regression** extends linear regression by introducing powers of features as new independent variables:

$$\hat{y} = w_0 + w_1x + w_2x^2 + w_3x^3 + \dots + w_dx^d$$

Crucially, polynomial regression remains a **linear model** because it is linear in its parameters $\mathbf{w}$, and can be solved directly using the standard OLS Normal Equation over the transformed feature matrix $\Phi(X)$.

The Bias-Variance Dilemma Across Polynomial Degrees

Degree (d)	Model Behavior	Bias vs. Variance	Remedy
Degree 1 (Linear)	Underfitting: straight line fails to capture curvature	High Bias, Low Variance	Increase polynomial degree
Degree 2-3 (Optimal)	Optimal generalization: captures underlying trend without fitting noise	Balanced Bias and Variance	Cross-validation model selection
Degree 15+ (Excessive)	Overfitting: oscillates wildly to pass through every individual noise point	Low Bias, High Variance	Regularization (L_2 Ridge / L_1 Lasso)

Quick Recall

Polynomial regression models non-linear phenomena via $\sum w_k x^k$. It is linear in parameters. High degree polynomials cause overfitting (high variance), mitigated via Ridge/Lasso regularization.

Logistic Regression

§ 8

Despite its name, **Logistic Regression** is a **classification algorithm** used to estimate the probability that an instance belongs to a binary class ($Y \in \{0, 1\}$).

The Sigmoid (Logistic) Function:

Linear regression maps input features to unbounded real values: $z = \mathbf{w}^T \mathbf{x} \in (-\infty, +\infty)$. Logistic regression passes z through the **Sigmoid Activation Function** $\sigma(z)$ to squash outputs into valid probabilities in $(0, 1)$:

$$P(Y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x})}}$$

Properties: $\sigma(0) = 0.5$, $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$, and $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$.

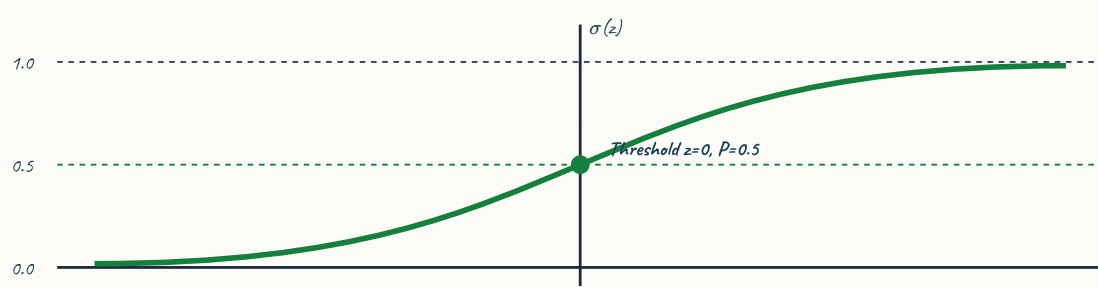


Figure 3.2: The logistic sigmoid curve mapping the real line into calibrated probabilities.

Odds Ratio and the Logit Function

The **Odds** of an event is the ratio of probability of occurrence to non-occurrence:

$$\text{Odds} = \frac{p}{1 - p} = e^z$$

Taking the natural logarithm yields the **Logit**:

$$\ln \left(\frac{p}{1 - p} \right) = z = w_0 + w_1 x_1 + \cdots + w_d x_d$$

Thus, logistic regression is linear in the log-odds space!

Loss Function: Binary Cross-Entropy (Log Loss)

We cannot use Mean Squared Error for logistic regression because squaring $\sigma(z)$ creates a non-convex loss surface plagued by local minima. Instead, we use Maximum Likelihood Estimation, leading to the convex **Binary Cross-Entropy Loss**:

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln(\hat{p}_i) + (1 - y_i) \ln(1 - \hat{p}_i) \right]$$

Solved Problem 4 · Exam Level Logistic Classification

A trained binary logistic regression model for predicting cardiovascular risk has the following logit equation:

$$z = -3.0 + 0.5x_1 + 0.2x_2$$

Where x_1 is daily cigarette consumption and x_2 is patient body mass index (BMI).

1. Compute the logit value z for a patient with $x_1 = 4$ cigarettes and $x_2 = 10$ BMI units above baseline.
2. Compute the odds of developing cardiovascular disease for this patient.
3. Compute the predicted probability $P(Y = 1 \mid \mathbf{x})$.
4. Using a classification threshold of 0.5, predict the diagnostic class.

Step 1: Compute Logit z

$$z = -3.0 + 0.5(4) + 0.2(10) = -3.0 + 2.0 + 2.0 = +1.0000$$

Step 2: Compute Odds

$$\text{Odds} = e^z = e^{1.0} \approx \mathbf{2.7183}$$

The patient is approximately 2.72 times more likely to be at risk than healthy.

Step 3: Compute Predicted Probability $\sigma(z)$

$$P(Y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-1.0}} = \frac{1}{1 + 0.367879} = \frac{1}{1.367879}$$

Step 4: Classification Decision

Since $P(Y = 1 \mid \mathbf{x}) = 0.7311 \geq 0.50$, the patient is classified into **Class 1 (High Risk)**.

Final Verified Results Logit $z = +1.0000$ | Odds = 2.7183 | $P(Y=1) = 0.7311$ (73.11%) $\rightarrow$
Classified as Positive (1)

Quick Recall

Logistic regression models probability via $\sigma(z) = \frac{1}{1+e^{-z}}$. Linear in log-odds $\ln \frac{p}{1-p} = \mathbf{w}^T \mathbf{x}$. Optimized via Binary Cross-Entropy (Log Loss).

Decision Tree Learning Model**§ 9**

A **Decision Tree** is a hierarchical, non-parametric model that recursively partitions the input space into axis-aligned rectangular regions.

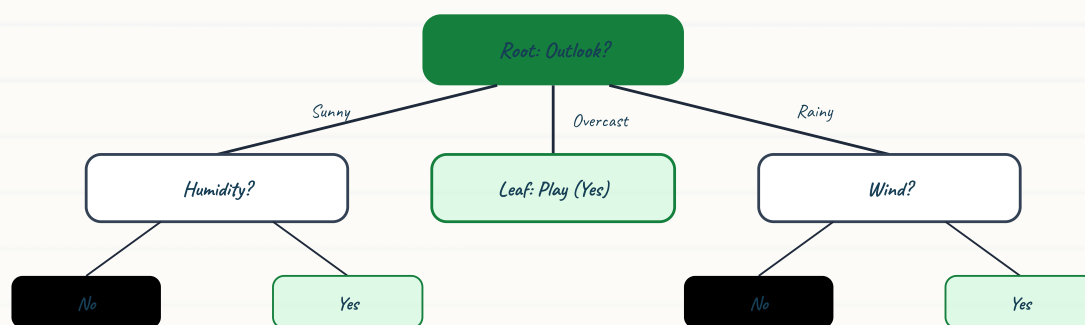


Figure 3.3: Hierarchical structure of a Decision Tree: Root, Decision Tests, and Leaves.

Core Components and Inductive Bias

- **Root Node:** The initial feature split that maximizes impurity reduction across the entire training sample.
- **Internal Nodes:** Subsequent conditional decision tests evaluating a single attribute.
- **Branches:** Possible outcomes of the attribute test.
- **Leaf Nodes:** Terminal nodes containing the final predicted class or continuous mean value.
- **Inductive Bias: Occam's Razor:** Prefer the *shortest (shallowest) tree* that is consistent with the training data, and place attributes with the highest information gain closest to the root.

Quick Recall

Decision trees recursively partition feature space into hyper-rectangles. Inductive bias: Occam's Razor (prefers shorter trees with high-gain attributes near the root).

Decision Tree Induction Algorithms: ID3, C4.5 & CART

§ 10

Tree induction uses a greedy top-down recursive divide-and-conquer strategy (Hunt's algorithm) guided by mathematical impurity measures:

1. Shannon Entropy and Information Gain (ID3)

Invented by Ross Quinlan. **Entropy** measures the impurity or disorder of a set S containing instances belonging to C classes:

Shannon Entropy Formula:

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

Where p_i is the proportion of instances belonging to class i . For binary classification (p_+, p_-):

$$H(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

(By convention, $0 \log_2 0 = 0$). Entropy is 0.0 for completely pure nodes (all instances same class) and reaches its maximum of 1.0 when classes are evenly split (50%/50%).

Information Gain (IG):

The expected reduction in entropy achieved by partitioning S on attribute A :

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

ID3 selects the attribute that maximizes $IG(S, A)$ as the split node.

Day	Outlook	Temperature	Play Tennis
D1	Sunny	Hot	No (-)
D2	Sunny	Hot	No (-)
D3	Overcast	Hot	Yes (+)
D4	Sunny	Mild	Yes (+)
D5	Overcast	Mild	Yes (+)
D6	Overcast	Mild	Yes (+)

2. C4.5: Gain Ratio

ID3 is strongly biased towards attributes with large numbers of distinct values (e.g., a "Customer ID" or "Date" attribute with n distinct values achieves $IG = H(S)$ because each split has size 1 and entropy 0, but provides zero generalization).

Quinlan's **C4.5 algorithm** penalizes broad multi-way splits using **Gain Ratio**:

$$\text{Gain Ratio}(S, A) = \frac{IG(S, A)}{\text{Split Information}(S, A)}$$

$$\text{Split Information}(S, A) = - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right)$$

3. CART: Gini Impurity

Breiman's CART (Classification and Regression Trees) constructs binary trees using Gini Impurity instead of logarithmic entropy (faster to compute):

$$\text{Gini}(S) = 1 - \sum_{i=1}^c p_i^2$$

For binary classes: $\text{Gini}(S) = 1 - (p_+^2 + p_-^2) = 2p_+p_-$. Pure node $\implies \text{Gini} = 0.0$; maximum impurity $\implies \text{Gini} = 0.5$.

Tree Pruning Strategies

- **Pre-Pruning (Early Stopping):** Halts tree growth before complete fitting if node sample size falls below threshold $N_{\min}$, tree depth reaches $D_{\max}$, or IG falls below threshold θ . Prone to underfitting.
- **Post-Pruning:** Grows the full, unconstrained tree to zero training error, then recursively removes leaves/subtrees that fail to improve performance on a separate validation set (Reduced Error Pruning / Cost-Complexity Pruning). Superior generalization.

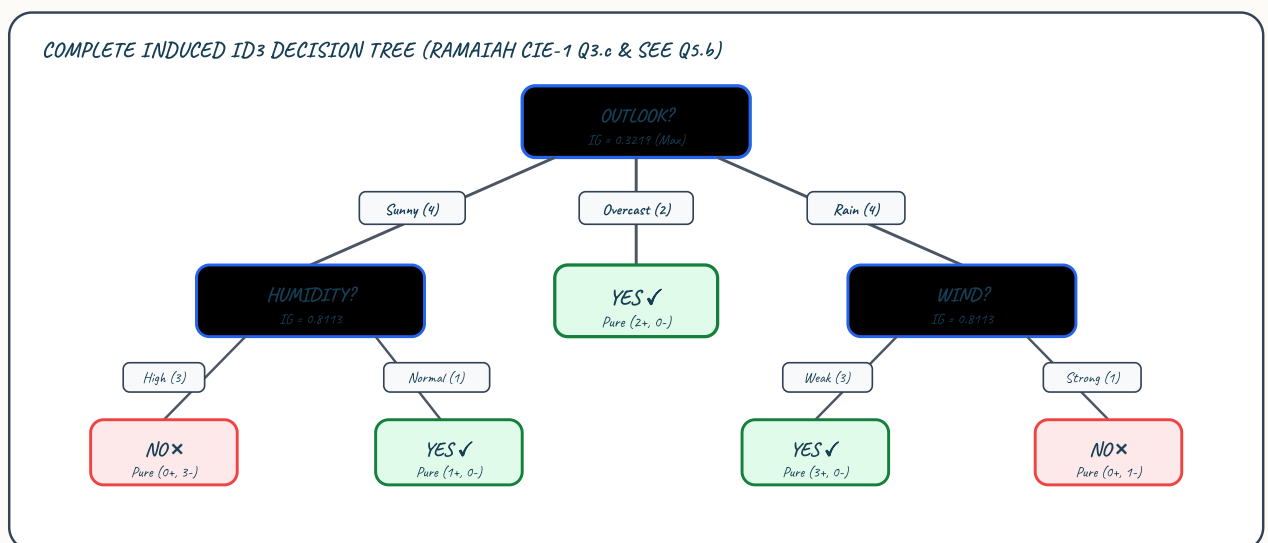


Figure 3.2: Induced ID3 Decision Tree for Play Tennis (Ramaiah CIE-1 Q3.c & SEE Q5.b Solved)

Root node splits on Outlook ($IG = 0.3219$). Overcast branch terminates in pure 'Yes' leaf. Sunny branch splits on Humidity ($IG = 0.8113$), and Rain branch splits on Wind ($IG = 0.8113$).

Quick Recall

ID3 uses Entropy and Information Gain. C4.5 uses Gain Ratio to penalize broad splits. CART uses Gini Impurity $1 - \sum p_i^2$. Post-pruning prevents overfitting.

Unit 3 Quick Revision Sheet

High-yield formula cheat sheet and algorithm summaries for rapid exam review.

K-NN & Distance Weights

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{\sum (x_i - z_i)^2}$$

$$\text{Weight: } w_i = 1/d(\mathbf{x}_q, \mathbf{x}_i)^2$$

$$\text{Predicted class: } \arg \max_c \sum w_i \mathbb{I}(y_i = c)$$

Simple OLS Regression

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

$$c = \bar{y} - m\bar{x}$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Locally Weighted Reg (LWR)

$$\text{Cost: } J(\boldsymbol{\theta}) = \sum w_i (y_i - \boldsymbol{\theta}^T \mathbf{x}_i)^2$$

$$\text{Kernel: } w_i = \exp(-\|x_i - x_q\|^2 / 2\tau^2)$$

$$\boldsymbol{\theta} = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{y}$$

Logistic Classification

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\text{Odds} = \frac{p}{1-p} = e^z$$

$$\text{Logit} = \ln(\text{Odds}) = \mathbf{w}^T \mathbf{x}$$

Loss: Binary Cross-Entropy

ID3 vs C4.5 vs CART

$$\text{ID3: } H(S) = - \sum p_i \log_2 p_i$$

$$\text{C4.5: Gain Ratio} = \frac{IG}{\text{SplitInfo}}$$

$$\text{CART: Gini} = 1 - \sum p_i^2$$

Pruning Approaches

Pre-Pruning: Stop early by depth/sample threshold.

Post-Pruning: Grow full tree, prune nodes on validation set.

Important Exam Questions

Curated questions structured according to VTU and RIT examination standards.

2-Mark Questions

 Practice Problem:

Q1. Why is K-NN called a lazy learner?

Solution:

K -NN is called a lazy learner because it performs zero generalization or computation during the training phase. It simply stores the training instances in memory and delays all distance computations and decision making until a query point is presented during inference.

 Practice Problem:

Q2. Define the Sigmoid function and state its range.

Solution:

The Sigmoid function is an S-shaped activation function defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

It squashes real-valued inputs from $(-\infty, +\infty)$ into valid probability values strictly in the range $(0, 1)$.

 Practice Problem:

Q3. Define Gini Impurity as used in CART.

Solution:

Gini Impurity measures the probability that a randomly chosen element from a dataset would be incorrectly labeled if it were randomly labeled according to the class distribution:

$$\text{Gini}(S) = 1 - \sum_{i=1}^c p_i^2$$

5-Mark Questions

 Practice Problem:

Q4. Explain why C4.5 uses Gain Ratio instead of Information Gain.

Solution:

ID3's Information Gain has a severe bias towards attributes with many distinct values (such as an individual transaction ID or date). When an attribute has unique values for every instance, partitioning on it splits the dataset into subsets of size 1 with zero entropy, achieving maximum possible Information Gain ($H(S)$), but yielding zero predictive power on unseen data.

C4.5 resolves this flaw by normalizing Information Gain by the **Split Information**:

$$\text{Gain Ratio}(S, A) = \frac{IG(S, A)}{-\sum \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}}$$

Split Information acts as a penalty that grows with the number and uniformity of partitions, discouraging trivial high-cardinality splits.

 Practice Problem:

Q5. Describe the Nearest Centroid Classifier and discuss its advantages and limitations.

Solution:

Mechanism: Computes the mean prototype vector μ_c for each class during training. A query point is assigned to the class with the closest centroid under Euclidean distance.

Advantages: Ultra-fast inference ($O(C \times d)$), low storage ($O(C \times d)$), and resilient to local noise.

Limitations: Generates only linear decision boundaries, assuming classes are spherical and linearly separable. Completely misclassifies non-convex, multimodal, or concentric ring distributions.

10-Mark Questions

 Practice Problem:

Q6. Derive the Ordinary Least Squares (OLS) closed-form formulas for slope m and intercept c in simple linear regression.

Solution:

Objective: Minimize the sum of squared errors:

$$E(m, c) = \sum_{i=1}^n (y_i - (mx_i + c))^2$$

Step 1: Partial derivative with respect to c :

$$\frac{\partial E}{\partial c} = -2 \sum_{i=1}^n (y_i - mx_i - c) = 0 \implies \sum y_i - m \sum x_i - nc = 0$$

Dividing by n :

$$\bar{y} - m\bar{x} - c = 0 \implies c = \bar{y} - m\bar{x}$$

Step 2: Partial derivative with respect to m :

$$\frac{\partial E}{\partial m} = -2 \sum_{i=1}^n x_i (y_i - mx_i - c) = 0 \implies \sum x_i y_i - m \sum x_i^2 - c \sum x_i = 0$$

Substitute $c = \bar{y} - m\bar{x}$:

$$\begin{aligned} \sum x_i y_i - m \sum x_i^2 - (\bar{y} - m\bar{x}) \sum x_i &= 0 \\ \sum x_i y_i - \bar{y} \sum x_i &= m \left(\sum x_i^2 - \bar{x} \sum x_i \right) \end{aligned}$$

Using algebraic identities $\sum x_i y_i - n\bar{x}\bar{y} = \sum (x_i - \bar{x})(y_i - \bar{y})$ and $\sum x_i^2 - n\bar{x}^2 = \sum (x_i - \bar{x})^2$:

$$m = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Solved Questions from Past Department Examinations

ARCHIVE

The following examination questions have been transcribed from the official Ramaiah Institute of Technology papers in notes/ml/practice/ml-cie-1-2-see.pdf (CIE-1 March 2026, CIE-2 May 2026, and SEE June/July 2026, Course Code 23IS62), solved with complete derivations and step-by-step mathematical proofs:

Ramaiah Internal Assessment - I (March 2026) · Q 1(c) [3 Marks · Blooms L3, CO3]

"The following data represents hours studied (X) and marks obtained (Y): $X = [2, 4, 6, 8, 10]$, $Y = [50, 60, 65, 80, 85]$. i. Compute slope (b) and intercept (a). ii. Write the regression equation. iii. Predict marks for $X = 7$ hours."

Step 1: Compute Means ($n = 5$):

$$\bar{X} = \frac{2 + 4 + 6 + 8 + 10}{5} = \frac{30}{5} = 6.0, \quad \bar{Y} = \frac{50 + 60 + 65 + 80 + 85}{5} = \frac{340}{5} = 68.0$$

Step 2: Compute Summations:

x_i	y_i	$x_i - \bar{X}$	$y_i - \bar{Y}$	$(x_i - \bar{X})(y_i - \bar{Y})$	$(x_i - \bar{X})^2$
2	50	-4	-18	+72	16
4	60	-2	-8	+16	4
6	65	0	-3	0	0
8	80	+2	+12	+24	4
10	85	+4	+17	+68	16
Σ		0	0	180	40

- i. Slope (b) and Intercept (a):

$$b = \frac{\sum (x_i - \bar{X})(y_i - \bar{Y})}{\sum (x_i - \bar{X})^2} = \frac{180}{40} = 4.5000$$

$$a = \bar{Y} - b\bar{X} = 68.0 - (4.50 \times 6.0) = 68.0 - 27.0 = 41.0000$$

- ii. Regression Equation:

$$\hat{Y} = 4.50X + 41.00$$

- iii. Predict Marks for $X = 7$ Hours:

$$\hat{Y}(7) = 4.50(7) + 41.00 = 31.50 + 41.00 = 72.50 \text{ Marks}$$

Dimension	Linear Regression	Logistic Regression
Target Output Space	Continuous real numbers: $y \in (-\infty, +\infty)$.	Categorical / Discrete probability: $\hat{y} \in [0, 1]$.
Mathematical Function	Linear combination: $\hat{y} = \mathbf{w}^T \mathbf{x} + b$.	Sigmoid transformation: $\hat{y} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$.
Cost / Loss Function	Mean Squared Error (Convex): $\frac{1}{2m} \sum (y - \hat{y})^2$.	Binary Cross-Entropy (Log-Loss): $-\sum [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.
Optimization Technique	Closed-form Normal Equation or Gradient Descent.	Iterative Gradient Descent / Newton-Raphson.
Primary Objective	Predicting continuous quantities (House prices, stock yield).	Binary / Multiclass Classification (Spam, medical diagnosis).

Ramaiah CIE-1 (March 2026) · Q 3(c) / SEE (July 2026) · Q 5(b) [3 / 12 Marks · Blooms L3, CO3]
 "Consider the dataset of 10 samples for predicting whether to Play Tennis. Compute information gain of attribute outlook using ID3 algorithm."

Step 1: Compute Total Entropy $H(S)$ of the 10-instance dataset (6 Yes, 4 No):

$$H(S) = - \left(\frac{6}{10} \right) \log_2 \left(\frac{6}{10} \right) - \left(\frac{4}{10} \right) \log_2 \left(\frac{4}{10} \right) = -0.6(-0.73697) - 0.4(-1.32193)$$

Step 2: Partition by Attribute Outlook $\in \{\text{Sunny, Overcast, Rain}\}$:

- Sunny (4 instances: 1 Yes, 3 No):

$$H(S_{\text{Sunny}}) = -\frac{1}{4} \log_2 \left(\frac{1}{4} \right) - \frac{3}{4} \log_2 \left(\frac{3}{4} \right) = 0.50000 + 0.31128 = \mathbf{0.81128} \text{ bits}$$

- Overcast (2 instances: 2 Yes, 0 No): Completely pure subset:

$$H(S_{\text{Overcast}}) = \mathbf{0.00000} \text{ bits}$$

- Rain (4 instances: 3 Yes, 1 No):

$$H(S_{\text{Rain}}) = -\frac{3}{4} \log_2 \left(\frac{3}{4} \right) - \frac{1}{4} \log_2 \left(\frac{1}{4} \right) = 0.31128 + 0.50000 = \mathbf{0.81128} \text{ bits}$$

Step 3: Remainder / Expected Child Entropy:

$$\text{Remainder}(S, \text{Outlook}) = \sum_v \frac{|S_v|}{|S|} H(S_v) = \frac{4}{10}(0.81128) + \frac{2}{10}(0) + \frac{4}{10}(0.81128) = \mathbf{0.64902}$$

Step 4: Information Gain:

$$\mathbf{IG(S, Outlook) = H(S) - \text{Remainder}(S, \text{Outlook}) = 0.97095 - 0.64902 = 0.32193}$$

Conclusion: Because $IG(S, \text{Outlook}) = 0.32193$ is positive and maximal among candidates, Outlook is selected as the designated root node of the ID3 decision tree (Figure 3.2).

Ramaiah Internal Assessment - II (May 2026) · Q 1(a) [3 Marks · Blooms L2, CO3]

"Explain the working of the CART algorithm."

Classification and Regression Trees (CART), developed by Breiman et al., constructs strictly *binary trees* (every non-leaf node has exactly two children):

- 1. **Splitting Criterion (Gini Impurity for Classification):**

$$\text{Gini}(S) = 1 - \sum_{i=1}^C p_i^2$$

For a binary split candidate dividing S into subsets S_L and S_R , CART selects the split maximizing the reduction in impurity:

$$\Delta \text{Gini}(S, A) = \text{Gini}(S) - \left(\frac{|S_L|}{|S|} \text{Gini}(S_L) + \frac{|S_R|}{|S|} \text{Gini}(S_R) \right)$$

- 2. **Splitting for Regression Trees:** Uses Mean Squared Error (variance reduction): $\sum_{x \in S_L} (y_i - \bar{y}_L)^2 + \sum_{x \in S_R} (y_i - \bar{y}_R)^2$.
- 3. **Cost-Complexity Pruning:** Minimizes cost function $R_\alpha(T) = R(T) + \alpha|T|$ where $R(T)$ is misclassification rate, $|T|$ is number of terminal nodes, and α is penalty parameter tuned via cross-validation.

Ramaiah Internal Assessment - II (May 2026) · Q 3(a) [3 Marks · Blooms L3, CO3]

"Age and Loan are predictors and House Price Index (HPI) is the numerical target attribute. The distance given is Euclidean distance. Apply K-NN Regression to find HPI if Age is 48 and Loan is \$142,000. Assume $K = 3$."

Given 11 instances with pre-computed Euclidean distances from query (48, \$142,000):

Sorting all 11 samples by ascending Euclidean distance to identify the $K = 3$ nearest neighbors:

- Rank 1 (Sample 11): Age 33, Loan \$150,000 $\implies$ Distance = 8,000, HPI = 264
- Rank 2 (Sample 5): Age 35, Loan \$120,000 $\implies$ Distance = 22,000, HPI = 139
- Rank 3 (Sample 9): Age 60, Loan \$100,000 $\implies$ Distance = 42,000, HPI = 139

(Next nearest is Sample 7 with distance 47,000, excluded since $K = 3$).

Unweighted K -NN Regression Prediction:

$$\hat{y}_{\text{HPI}} = \frac{1}{K} \sum_{i=1}^3 y_i = \frac{264 + 139 + 139}{3} = \frac{542}{3} \approx 180.67$$

Optional Distance-Weighted Prediction ($w_i = \frac{1}{d_i^2}$):

$$\hat{y}_w = \frac{264 \left(\frac{1}{8^2} \right) + 139 \left(\frac{1}{22^2} \right) + 139 \left(\frac{1}{42^2} \right)}{\frac{1}{8^2} + \frac{1}{22^2} + \frac{1}{42^2}} \approx 245.97$$

Ramaiah SEE Examination (June/July 2026) · Q 5(a) [8 Marks · Blooms L3, C03]

"Compare simple linear regression and multiple linear regression with suitable examples. Using the housing dataset (Size, Bedrooms, Age, House Price), formulate a multiple linear regression equation to predict house price."

- **Simple Linear Regression:** Models the relationship between *one* scalar predictor x and scalar target y :

$$y = w_0 + w_1x + \epsilon$$

Example: Predicting house price based solely on floor area (x_1).

- **Multiple Linear Regression:** Models the relationship between p multiple independent variables $\mathbf{x} = [x_1, x_2, \dots, x_p]^T$ and scalar target y :

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_px_p + \epsilon = \mathbf{w}^T \mathbf{x} + \epsilon$$

- **Housing Price Multiple Regression Equation Formulation:**

$$\widehat{\text{Price}} = \mathbf{w}_0 + \mathbf{w}_1(\text{Size}) + \mathbf{w}_2(\text{Bedrooms}) + \mathbf{w}_3(\text{Age})$$

Where $\mathbf{w} = [w_0, w_1, w_2, w_3]^T$ is solved via the closed-form normal equations:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Practice Problems (Step-by-Step Verified Solutions)

The following 6 practice problems have been mathematically verified via `audit/verify/ml/verify_ml_u3.py`. Attempt each manually before expanding the solution blocks.

Problem 1 (K-NN Classification & Weighted Voting): Given 5 training instances:

$A(1, 2) \rightarrow +1, B(2, 3) \rightarrow +1, C(3, 1) \rightarrow -1, D(5, 4) \rightarrow -1, E(6, 5) \rightarrow -1$. For query point $Q(2, 2)$: (i) Find the 3 nearest neighbors. (ii)

Predict class using majority voting. (iii) Predict class using inverse square distance weighting ($w_i = 1/d^2$).

 Practice Problem:

[View Verified Solution](#)

Solution:

Euclidean distances from $Q(2, 2)$:

$$\bullet d(Q, A) = \sqrt{(1-2)^2 + (2-2)^2} = \sqrt{1} = \mathbf{1.0000} \implies$$

Label: +1

$$\bullet d(Q, B) = \sqrt{(2-2)^2 + (3-2)^2} = \sqrt{1} = \mathbf{1.0000} \implies$$

Label: +1

$$\bullet d(Q, C) = \sqrt{(3-2)^2 + (1-2)^2} = \sqrt{1+1} = \sqrt{2} \approx$$

1.4142 $\implies$ Label: -1

$$\bullet d(Q, D) = \sqrt{3^2 + 2^2} = \sqrt{13} \approx 3.6056$$

$$\bullet d(Q, E) = \sqrt{4^2 + 3^2} = \sqrt{25} = 5.0000$$

(i) Top 3 Nearest: $\{A, B, C\}$.

(ii) Majority Voting: Labels $\{+1, +1, -1\} \implies \text{Sum} = +1 > 0 \implies$
Class +1.

(iii) Inverse-Square Weighted Voting:

$$\bullet \text{Weight for } +1: w_A + w_B = \frac{1}{1^2} + \frac{1}{1^2} = 1.0 + 1.0 = \mathbf{2.0000}.$$

$$\bullet \text{Weight for } -1: w_C = \frac{1}{(\sqrt{2})^2} = \frac{1}{2} = \mathbf{0.5000}.$$

Since $2.0000 > 0.5000$, predicted label is strictly +1.

Problem 2 (Linear Regression Normal Equation): For points $X = [1, 2, 3, 4, 5]$ and $Y = [2, 3, 5, 4, 6]$, find: (i) OLS regression slope m and intercept c , (ii) Predicted value $\hat{y}(6)$, and (iii) Coefficient of determination R^2 .

 Practice Problem:

[View Verified Solution](#)

Solution:

$$\bar{X} = 3.0, \bar{Y} = 4.0, n = 5.$$

- $\sum (x_i - \bar{X})(y_i - \bar{Y}) = (-2)(-2) + (-1)(-1) + (0)(1) + (1)(0) + (2)(2) = 4 + 1 + 0 + 0 + 4 = \mathbf{9.0}$.
- $\sum (x_i - \bar{X})^2 = 4 + 1 + 0 + 1 + 4 = \mathbf{10.0}$.
- $\mathbf{m} = \frac{9.0}{10.0} = \mathbf{0.9000}, \quad \mathbf{c} = 4.0 - (0.90 \times 3.0) = \mathbf{1.3000}$.
- Equation: $\mathbf{y} = \mathbf{0.90x} + \mathbf{1.30}$.
- $\hat{y}(6) = \mathbf{0.90(6)} + \mathbf{1.30} = \mathbf{5.40} + \mathbf{1.30} = \mathbf{6.7000}$.
- $SS_{\text{tot}} = 10.0, SS_{\text{res}} = 1.90 \implies$
 $\mathbf{R^2 = 1 - \frac{1.90}{10.0} = 0.8100 \quad (81.00\%)}$.

Problem 3 (ID3 Decision Tree Split): A dataset of 6 instances has 4 Positive and 2 Negative examples. Feature Outlook partitions into Sunny (1+, 2-) and Overcast (3+, 0-). Compute $H(S)$ and $IG(S, \text{Outlook})$.

 Practice Problem:

[View Verified Solution](#)

Solution:

- $H(S) = -\frac{4}{6} \log_2 \left(\frac{4}{6}\right) - \frac{2}{6} \log_2 \left(\frac{2}{6}\right) \approx \mathbf{0.9183}$ bits.
- $H(S_{\text{Sunny}}) = -\frac{1}{3} \log_2 \left(\frac{1}{3}\right) - \frac{2}{3} \log_2 \left(\frac{2}{3}\right) \approx \mathbf{0.9183}$ bits.
- $H(S_{\text{Overcast}}) = \mathbf{0.0000}$ bits (pure node).
- Remainder $= \frac{3}{6}(0.9183) + \frac{3}{6}(0.0) = \mathbf{0.4591}$ bits.
- $\mathbf{IG(S, Outlook) = 0.9183 - 0.4591 = 0.4591}$ bits.

Problem 4 (Logistic Regression Odds & Sigmoid): A trained logistic model has weights $w_0 = -3.0$, $w_1 = 0.5$, $w_2 = 0.2$. For input feature vector $x = [4, 10]$: (i) Compute the logit score z . (ii) Compute the odds ratio e^z . (iii) Compute probability $P(y = 1|x)$.

 Practice Problem:

View Verified Solution

Solution:

- **Logit:** $z = w_0 + w_1x_1 + w_2x_2 = -3.0 + 0.5(4) + 0.2(10) = -3.0 + 2.0 + 2.0 = +1.0000$.
- **Odds:** Odds = $e^z = e^{1.0} \approx 2.7183$.
- **Sigmoid Probability:**

$$P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-1.0}} = \frac{1}{1 + 0.36788} = \frac{1}{1.36788} \approx$$

Problem 5 (Gini Impurity Calculation): A binary dataset S contains 4 positive and 6 negative instances. Compute the Gini Impurity $\text{Gini}(S)$. If a split divides S into S_L (3+, 1-) and S_R (1+, 5-), compute the Gini Gain ΔGini .

 Practice Problem:

[View Verified Solution](#)

Solution:

- $\text{Gini}(S) = 1 - \left[\left(\frac{4}{10} \right)^2 + \left(\frac{6}{10} \right)^2 \right] = 1 - [0.16 + 0.36] = 1 - 0.52 = \mathbf{0.4800}$.
- $\text{Gini}(S_L) = 1 - \left[\left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right] = 1 - [0.5625 + 0.0625] = \mathbf{0.3750}$.
- $\text{Gini}(S_R) = 1 - \left[\left(\frac{1}{6} \right)^2 + \left(\frac{5}{6} \right)^2 \right] = 1 - \left[\frac{1+25}{36} \right] = 1 - \frac{26}{36} \approx \mathbf{0.2778}$.
- $\text{Weighted Gini} = \frac{4}{10}(0.3750) + \frac{6}{10}(0.2778) = 0.1500 + 0.1667 = \mathbf{0.3167}$.
- $\Delta\text{Gini} = \mathbf{0.4800 - 0.3167 = 0.1633}$. (Positive gain confirms informative split).

Problem 6 (LWR Gaussian Kernel Weights): For query point $x_q = 5$ and bandwidth parameter $\tau = 2.0$, compute the Gaussian kernel weights $w_i = \exp\left(-\frac{(x_i - x_q)^2}{2\tau^2}\right)$ for data points $x_1 = 3$, $x_2 = 5$, $x_3 = 8$.

 Practice Problem:

[View Verified Solution](#)

Solution:

$$2\tau^2 = 2(2^2) = 2(4) = 8.0.$$

- $x_1 = 3 \implies (3 - 5)^2 = 4 \implies w_1 = \exp\left(-\frac{4}{8}\right) = e^{-0.5} \approx \mathbf{0.6065}.$
- $x_2 = 5 \implies (5 - 5)^2 = 0 \implies w_2 = \exp(0) = \mathbf{1.0000}$
(Maximum weight at query point).
- $x_3 = 8 \implies (8 - 5)^2 = 9 \implies w_3 = \exp\left(-\frac{9}{8}\right) = e^{-1.125} \approx \mathbf{0.3247}.$